

LAB: Timer & PWM (~2024)

Servo motor and DC motor

Date: 2025-10-24

Author/Partner: Dongjun, Lee

Github: <https://github.com/henny041520-commits/EC-DJLee-042>

Demo Video: Youtube link

Problem 1: RC servo motor : <https://youtube.com/shorts/VR7RJFKOLqk?feature=share>

Problem 2: DC motor : <https://youtube.com/shorts/Lzfa8ZyxOhs?feature=share>

PDF version: [https://github.com/henny041520-commits/EC-DJLee-042/blob/main/report/LAB_Timer_%26_PWM_\(~2024\).pdf](https://github.com/henny041520-commits/EC-DJLee-042/blob/main/report/LAB_Timer_%26_PWM_(~2024).pdf)

Introduction

Create a simple program that control a servo motor and a DC motor with PWM output.

You must submit

- LAB Report (*.md & *.pdf)
- Zip source files(main*.c, ecRCC.h, ecGPIO.h, ecSysTick.c etc...).
- Only the source files. Do not submit project files

Requirement

Hardware

- MCU
 - NUCLEO-F411RE
- Actuator/Sensor/Others:
 - 3 LEDs and load resistance
 - RC Servo Motor (SG90)
 - DC motor (5V)
 - DC motor driver(LS9110s)
 - breadboard

Software

- Keil uVision, CMSIS, EC_HAL library

Tutorial: STM-Arduino

{% embed url="<https://ykkim.gitbook.io/ec/ec-course/tutorial/tutorial-arduino-stm32/tutorial-arduino-stm32-part-2#pwm-pulse-width-modulation-dc-motor>" %}

We are going to create a simple program that run DC - Motor by giving pwm signal as input.

Press the reset button(black) and verify the operation. If you press the user button, DC-Motor will turn on.

Procedure

1. Create a new project under the directory \EC\lab\LAB_PWM
2. Open *Arduino IDE* and Create a new program named as 'TU_arduino_PWM.ino'.
3. Write the following code.
4. upload and run.

```
const int pwmPin = 11;    // PWM pin
const int buttonPin = 3;  // button pin

int buttonState = HIGH;

void setup() {
    pinMode(pwmPin, OUTPUT);

    // initialize the pushbutton pin as an input:
    pinMode(buttonPin, INPUT_PULLUP);
    attachInterrupt(digitalPinToInterrupt(buttonPin), motorOperation, CHANGE);
}

void loop() {

    if (buttonState == LOW){
        for (int i = 0; i < 10; i++){
            analogWrite(pwmPin, 40 + 10*i);
            delay(100);
        }

        for (int i = 10; i > 0; i--){
            analogWrite(pwmPin, 40 + 10*i);
            delay(100);
        }
    }
}
```

```
else{
    analogWrite(pwmPin, 0);
}
}

void motorOperation(){
    buttonState = digitalRead(buttonPin);
}
```

Tutorial: STM32F4xx

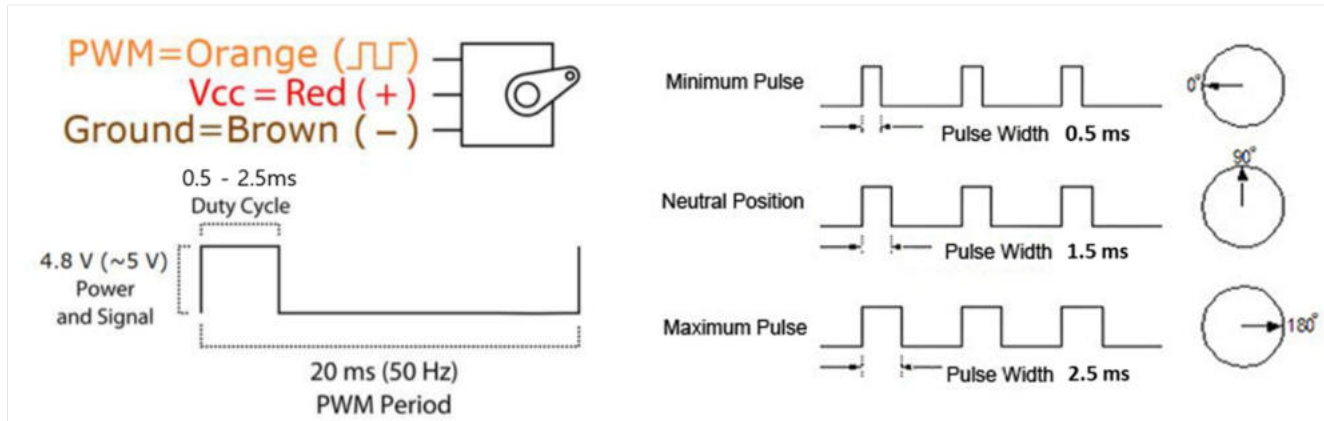
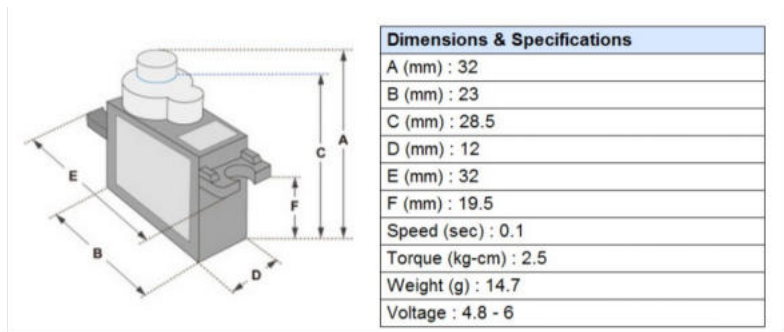
1. Tutorial: DC motor driver connection

Understand how to connect a DC motor to the motor driver (LS9110s).

{% embed url="<https://ykkim.gitbook.io/ec/ec-course/tutorial/tutorial-dcmotor-motor-driver-connection>" %}

Problem 1: RC servo motor

An RC servo motor is a tiny and light weight motor with high output power. It is used to control rotation angles, approximately 180 degrees (90 degrees in each direction) and commonly applied in RC car, and Small-scaled robots. The angle of the motor can be controlled by the pulse width (duty ratio) of PWM signal. The PWM period should be set at **20ms or 50Hz**. Refer to the datasheet of the RC servo motor for detailed specifications.



1-1. Create HAL library

Download files:

- [ecPinNames.h](#) [ecPinNames.c](#)
- [ecTIM_student.h](#), [ecTIM_student.c](#)
- [ecPWM_student.h](#), [ecPWM_student.c](#)

Then, change the library files as

- ecTIM.h, ecTIM.c
- ecPWM.h, ecPWM.c

Declare and define the following functions in your library. You must update your header files located in the directory EC \include\.

ecTIM.h

```
// Timer Period setup
void TIM_init(TIM_TypeDef *TIMx, uint32_t msec);
void TIM_period(TIM_TypeDef* TIMx, uint32_t msec);
void TIM_period_ms(TIM_TypeDef* TIMx, uint32_t msec);
void TIM_period_us(TIM_TypeDef* TIMx, uint32_t usec);

// Timer Interrupt setup
void TIM_UI_init(TIM_TypeDef* TIMx, uint32_t msec);
```

```

void TIM_UI_enable(TIM_TypeDef* TIMx);
void TIM_UI_disable(TIM_TypeDef* TIMx);

// Timer Interrupt Flag
uint32_t is_UIF(TIM_TypeDef *TIMx);
void clear_UIF(TIM_TypeDef *TIMx);

```

ecTIM.c

```

/**
*****
* Detailed Description:
*   This file implements basic Timer initialization, period setting (µs / ms),
*   and update interrupt configuration for STM32F4xx timers.
*
*   TIMx counts up from 0 to ARR (Auto-Reload Register) with a counter clock
*   frequency determined by the system clock and prescaler (PSC).
*
*   When the counter overflows (reaches ARR), the Update Interrupt Flag (UIF)
*   is set, and the ISR (e.g., TIMx_IRQHandler) is executed.
*
*   The timer frequency can be adjusted by:
*       PSC: divides input clock
*       ARR: determines counting period (overflow rate)
*
*****
*/

#include "ecTIM2.h"
#include "ecGPIO2.h"

/* -----
* Function: TIM_init
* Purpose : Enable timer peripheral clock, set default 1 ms update event,
*           configure as up-counter, and start counting.
*
* Formula : Counter frequency = f_TIM / (PSC + 1)
*           Update event freq = Counter frequency / (ARR + 1)
*
* Default : PSC and ARR are set to generate 1 ms update interval.
* -----
*/

void TIM_init(TIM_TypeDef* TIMx){
    // 1. Enable corresponding peripheral clock in RCC
    if(TIMx == TIM1)        RCC->APB2ENR |= RCC_APB2ENR_TIM1EN;

```

```

else if(TIMx == TIM2) RCC->APB1ENR |= 1<<0;
else if(TIMx == TIM3) RCC->APB1ENR |= 1<<1;
else if(TIMx == TIM4) RCC->APB1ENR |= 1<<2;
else if(TIMx == TIM5) RCC->APB1ENR |= 1<<3;
else if(TIMx == TIM9) RCC->APB2ENR |= 1<<16;
else if(TIMx == TIM11)RCC->APB2ENR |= 1<<18;

// 2. Set timer period to 1 ms (default)
uint32_t msec = 1;
TIM_period_ms(TIMx, msec);

// 3. Configure counter direction (DIR = 0 → upcounting)
TIMx->CR1 &= ~(1<<4);

// 4. Enable the counter (CEN bit)
TIMx->CR1 |= TIM_CR1_CEN;
}

/* -----
 * Function: TIM_period_us
 * Purpose : Set timer update event period in microseconds.
 * Range    : 1 μs ~ 6000 μs
 *
 * Logic     :
 * - The function configures the prescaler (PSC) to create a 1 MHz counter
 *   clock (1 tick = 1 μs).
 * - ARR is then set equal to the number of microseconds (usec).
 * - When CNT reaches ARR, an update event occurs every <usec> μs.
 *
 * Example   :
 * For Sys_CLK = 84 MHz → PSC = 84 → f_cnt = 1 MHz → ARR = 1000 → 1 ms
period
 * -----
 */
void TIM_period_us(TIM_TypeDef *TIMx, uint32_t usec){
    uint16_t PSCval;
    uint32_t Sys_CLK;

    // 1. Detect current system clock source (PLL = 84 MHz, HSI = 16 MHz)
    if((RCC->CFGR & RCC_CFGR_SW_PLL) == RCC_CFGR_SW_PLL)
        Sys_CLK = 84000000;
    else if((RCC->CFGR & RCC_CFGR_SW_HSI) == RCC_CFGR_SW_HSI)
        Sys_CLK = 16000000;

    // 2. Calculate prescaler for 1 MHz timer counter
    PSCval = Sys_CLK / 1000000;    // e.g., 84 or 16 → f_cnt = 1 MHz

```

```

// 3. Compute ARR for required usec period
if (TIMx == TIM2 || TIMx == TIM5){ // 32-bit timers (larger ARR)
    uint32_t ARRval = (Sys_CLK / PSCval / 1000000) * usec; // 1 tick = 1
µs
    TIMx->PSC = PSCval - 1;
    TIMx->ARR = ARRval - 1;
} else { // 16-bit timers
    uint16_t ARRval = (uint16_t)((Sys_CLK / PSCval / 1000000) * usec);
    TIMx->PSC = PSCval - 1;
    TIMx->ARR = ARRval - 1;
}
}

/* -----
* Function: TIM_period_ms
* Purpose : Set timer update event period in milliseconds.
* Range   : 1 ms ~ 600 ms
*
* Logic   :
* - For TIM2/TIM5 (32-bit): counter = 100 kHz (PSC = Sys_CLK/1000000)
*   → ARR = 100 * msec
* - For other timers (16-bit): counter = 1 kHz (PSC = Sys_CLK/1000)
*   → ARR = 1 * msec
*
* Example (TIM3, 84 MHz clock):
* PSC = 84000 - 1 → f_cnt = 1 kHz
* ARR = 500 - 1 → Update event every 500 ms
* -----
*/
void TIM_period_ms(TIM_TypeDef* TIMx, uint32_t msec){
    uint16_t PSCval;
    uint32_t Sys_CLK;

    // 1. Determine system clock frequency
    if((RCC->CFGR & RCC_CFGR_SW_PLL) == RCC_CFGR_SW_PLL)
        Sys_CLK = 84000000;
    else if((RCC->CFGR & RCC_CFGR_SW_HSI) == RCC_CFGR_SW_HSI)
        Sys_CLK = 16000000;

    // 2. Configure PSC and ARR
    if (TIMx == TIM2 || TIMx == TIM5){
        // For 32-bit timers: counter clock = 100 kHz
        uint32_t ARRval;
        PSCval = Sys_CLK / 100000; // PSC gives 100 kHz
counter
    }
}

```

```

        ARRval = (Sys_CLK / PSCval / 1000) * msec;    // ARR = 100 * msec
        TIMx->PSC = PSCval - 1;
        TIMx->ARR = ARRval - 1;
    } else {
        // For 16-bit timers: counter clock = 1 kHz
        uint16_t ARRval;
        PSCval = Sys_CLK / 1000;                      // PSC gives 1 kHz
counter
        ARRval = (uint16_t)((Sys_CLK / PSCval / 1000) * msec); // ARR = 1 *
msec
        TIMx->PSC = PSCval - 1;
        TIMx->ARR = ARRval - 1;
    }
}

/* -----
 * Function: TIM_period
 * Purpose : Wrapper for ms-based period setting.
 * -----
 */
void TIM_period(TIM_TypeDef* TIMx, uint32_t msec){
    TIM_period_ms(TIMx, msec);
}

/* -----
 * Function: TIM_UI_init
 * Purpose : Configure the timer to generate an Update Event Interrupt (UIF).
 *
 * Procedure:
 *   1. Initialize timer (clock enable, upcount, enable counter)
 *   2. Enable update interrupt (DIER.UIE)
 *   3. Enable NVIC for corresponding timer interrupt channel
 *
 * Note:
 *   - The 'msec' parameter is not used here; period must be set separately.
 * -----
 */
void TIM_UI_init(TIM_TypeDef* TIMx, uint32_t msec){
    TIM_init(TIMx);                // Basic timer setup
    TIM_UI_enable(TIMx);           // Enable update interrupt in DIER
    TIM_period_ms(TIMx, msec);
    // 3. NVIC channel mapping per timer instance
    uint32_t IRQn_reg = 0;
    if(TIMx == TIM1)               IRQn_reg = TIM1_UP_TIM10_IRQn;
    else if(TIMx == TIM2)          IRQn_reg = TIM2_IRQn;
    else if(TIMx == TIM3)          IRQn_reg = TIM3_IRQn;

```

```

    else if(TIMx == TIM4)  IRQn_reg = TIM4_IRQn;
    else if(TIMx == TIM5)  IRQn_reg = TIM5_IRQn;
    else if(TIMx == TIM9)  IRQn_reg = TIM1_BRK_TIM9_IRQn;
    else if(TIMx == TIM10) IRQn_reg = TIM1_UP_TIM10_IRQn;
    else if(TIMx == TIM11) IRQn_reg = TIM1_TRG_COM_TIM11_IRQn;

    // Enable NVIC interrupt and set priority (lower number = higher priority)
    NVIC_EnableIRQ(IRQn_reg);
    NVIC_SetPriority(IRQn_reg, 2);
}

/* -----
 * Function: TIM_UI_enable
 * Purpose : Enable timer update interrupt generation.
 * Register: DIER.UIE = 1
 * -----
 */
void TIM_UI_enable(TIM_TypeDef* TIMx){
    TIMx->DIER |= 1<<0;
}

/* -----
 * Function: TIM_UI_disable
 * Purpose : Disable timer update interrupt generation.
 * Register: DIER.UIE = 0
 * -----
 */
void TIM_UI_disable(TIM_TypeDef* TIMx){
    TIMx->DIER &= ~(1<<0);
}

/* -----
 * Function: is_UIF
 * Purpose : Check Update Interrupt Flag (UIF) status.
 * Return  : 1 if set (overflow occurred), else 0.
 * Register: SR bit 0 (UIF)
 * -----
 */
uint32_t is_UIF(TIM_TypeDef *TIMx){
    return (TIMx->SR & 1);
}

/* -----
 * Function: clear_UIF
 * Purpose : Clear Update Interrupt Flag (UIF) by writing 0 to SR.UIF.
 * -----

```

```

*/
void clear_UIF(TIM_TypeDef *TIMx){
    TIMx->SR &= ~1;
}

```

ecPWM.h

```

/* PWM Configuration using PinName_t Structure */

/* PWM initialization */
// Default: 84MHz PLL, 1MHz CK_CNT, 50% duty ratio, 1msec period
void PWM_init(PinName_t pinName);
void PWM_pinmap(PinName_t pinName, TIM_TypeDef **TIMx, int *chN);

/* PWM PERIOD SETUP */
// allowable range for msec: 1~2,000
void PWM_period(PinName_t pinName, uint32_t msec);
void PWM_period_ms(PinName_t pinName, uint32_t msec); // same as
PWM_period()
// allowable range for usec: 1~1,000
void PWM_period_us(PinName_t pinName, uint32_t usec);

/* DUTY RATIO SETUP */
// High Pulse width in msec
void PWM_pulsewidth(PinName_t pinName, uint32_t pulse_width_ms);
void PWM_pulsewidth_ms(PinName_t pinName, uint32_t pulse_width_ms); // same
as void PWM_pulsewidth
// Duty ratio 0~1.0
void PWM_duty(PinName_t pinName, float duty);

```

ecPWM.c

```

/**
*****
* This module provides PWM initialization and control helpers using PinName_t.
* It configures GPIO in AF mode, maps a pin to TIMx and channel, sets PWM
mode,
* and exposes period/pulsewidth/duty APIs that write ARR/PSC/CCR registers.
*****
*/

```

```

#include "stm32f4xx.h"    // CMSIS device header for STM32F4: peripheral
register defs
#include "ecPWM2.h"        // Course HAL header: PinName_t, ecPinmap, PWM
helpers
#include "math.h"          // (Not strictly used here; kept as original include)

/* PWM Configuration using PinName_t Structure */

/* PWM initialization */
// Default: 84MHz PLL, 1MHz CK_CNT, 50% duty ratio, 1msec period
void PWM_init(PinName_t pinName){

// 0. Match TIMx from Port and Pin
//    - ecPinmap: resolve logical PinName_t to (GPIO port pointer, pin number)
//    - PWM_pinmap: resolve pin to owning TIM instance and channel number
    GPIO_TypeDef *port;                // GPIO port base address where
    'pinName' lives
    unsigned int pin;                  // GPIO pin number (0..15)
    ecPinmap(pinName, &port, &pin);    // Fill 'port' and 'pin' for this
    PinName_t
    TIM_TypeDef *TIMx;                 // Timer instance associated with this
    pin
    int chN;                           // Timer channel index (1..4) for this
    pin
    PWM_pinmap(pinName, &TIMx, &chN); // Resolve (TIMx, chN) from pin
    mapping

// 1. Initialize GPIO port and pin as AF
//    - Put the pin into Alternate Function mode so the timer can drive it.
//    - The actual AF number is set via AFR registers below.
    GPIO_init(pinName, AF); // AF mode (Alternate Function). Comment says AF=2
    but actual AF is set by AFR next.

    // GPIO_otype(port, pin, EC_PUSH_PULL); // Optional: force push-pull
    output type if needed
    // GPIO_pupdp(port, pin, EC_PU);         // Optional: enable internal
    pull-up if needed

// 2. Configure GPIO AFR by Pin num.
//    - Each GPIO pin requires selecting the correct AF (Alternate Function)
    number.
//    - AFR[0] covers pins 0..7, AFR[1] covers pins 8..15 (4 bits per pin).
//    - For STM32F4: TIM1/TIM2 generally map to AF1, TIM3/4/5 to AF2 on their

```

```

valid pins.
// - Below sets AF1 or AF2 by OR-ing the bitfield for this 'pin'.
//   AFR[0] for pin: 0~7,      AFR[1] for pin 8~15
//   AFR=1 for TIM1,TIM2  AFR=2 for TIM3 etc
if((TIMx == TIM1) || (TIMx == TIM2)){
    port->AFR[pin >>3] |= 0x01 <<(4*(pin % 8)); // Select AF1 for this pin
(TIM1/TIM2 output)
}
else if(TIMx == TIM3 || TIMx==TIM4 || TIMx==TIM5)
{
    port->AFR[pin>>3] |= 0x02<<(4*(pin % 8)); // Select AF2 for this pin
(TIM3/TIM4/TIM5 output)
}

// 3. Initialize Timer
// - TIM_init: enables timer clock, sets a default period (1 ms in this
HAL), upcounter, then starts it.
// - We immediately stop the counter (clear CEN) to safely reconfigure PWM
settings below.
    TIM_init(TIMx); // Base timer init (HAL default 1
ms period)
    TIMx->CR1 &= ~TIM_CR1_CEN; // Stop counter during
configuration

// 3-2. Direction of Counter
// - Ensure timer runs in up-counting mode (DIR=0). Down-counting would
invert PWM timing.
//YOUR CODE GOES HERE
    TIMx->CR1 &= ~TIM_CR1_DIR; // 0 = up-counting, 1 = down-
counting

// 4. Configure Timer Output mode as PWM
// - We configure "PWM mode 1" for the resolved channel.
// - Default duty = 50% by writing CCRx = ARR/2.
// - CCER polarity set to active-high; channel output enabled.
    uint32_t ccVal = TIMx->ARR/2; // Default compare = half of ARR
⇒ 50% duty
    if(chN == 1){
        TIMx->CCMR1 &= ~TIM_CCMR1_OC1M; // Clear OC1M bits
(channel 1 mode)
        TIMx->CCMR1 |= TIM_CCMR1_OC1M_1 | TIM_CCMR1_OC1M_2; // OC1M=110 (PWM
mode 1) for CH1
        TIMx->CCMR1 |= TIM_CCMR1_OC1PE; // CH1 preload
enable (buffer CCR1 until update)

```

```

        TIMx->CCR1 = ccVal; // Set CCR1 to 50%
duty
        TIMx->CCER &= ~TIM_CCER_CC1P; // Active high
polarity on CH1
        TIMx->CCER |= TIM_CCER_CC1E; // Enable CH1
output
    }
    else if(chN == 2){
        TIMx->CCMR1 &= ~TIM_CCMR1_OC2M; // Clear OC2M bits
(channel 2 mode)
        TIMx->CCMR1 |= TIM_CCMR1_OC2M_1 | TIM_CCMR1_OC2M_2; // OC2M=110 (PWM
mode 1) for CH2
        TIMx->CCMR1 |= TIM_CCMR1_OC2PE; // CH2 preload
enable
        TIMx->CCR1 = ccVal; // NOTE: writes
CCR1 per original code (kept unchanged)
        TIMx->CCER &= ~TIM_CCER_CC2P; // Active high
polarity on CH2
        TIMx->CCER |= TIM_CCER_CC2E; // Enable CH2
output
    }
    else if(chN == 3){
        TIMx->CCMR2 &= ~TIM_CCMR2_OC3M; // Clear OC3M bits
(channel 3 mode)
        TIMx->CCMR2 |= TIM_CCMR2_OC3M_1 | TIM_CCMR2_OC3M_2; // OC3M=110 (PWM
mode 1) for CH3
        TIMx->CCMR2 |= TIM_CCMR2_OC3PE; // CH3 preload
enable
        TIMx->CCR1 = ccVal; // NOTE: writes
CCR1 per original code (kept unchanged)
        TIMx->CCER &= ~TIM_CCER_CC3P; // Active high
polarity on CH3
        TIMx->CCER |= TIM_CCER_CC3E; // Enable CH3
output
    }
    else if(chN == 4){
        TIMx->CCMR2 &= ~TIM_CCMR2_OC4M; // Clear OC4M bits
(channel 4 mode)
        TIMx->CCMR2 |= TIM_CCMR2_OC4M_1 | TIM_CCMR2_OC4M_2; // OC4M=110 (PWM
mode 1) for CH4
        TIMx->CCMR2 |= TIM_CCMR2_OC4PE; // CH4 preload
enable
        TIMx->CCR1 = ccVal; // NOTE: writes
CCR1 per original code (kept unchanged)
        TIMx->CCER &= ~TIM_CCER_CC4P; // Active high
polarity on CH4

```

```

        TIMx->CCER |= TIM_CCER_CC4E;           // Enable CH4
output
    }

// 5. Enable Timer Counter
//    - For advanced timer TIM1, MOE (Main Output Enable) must be set to drive
//      outputs.
//    - Finally start the counter (CEN=1).
//      For TIM1 ONLY
    if(TIMx == TIM1) TIMx->BDTR |= TIM_BDTR_MOE;           // Enable main
output for TIM1
    // Enable timers
    TIMx->CR1 |= TIM_CR1_CEN;           // Start timer
counter
}

/* PWM PERIOD SETUP */
// allowable range for msec: 1~2,000
void PWM_period_ms(PinName_t pinName, uint32_t msec){

// 0. Match TIMx from Port and Pin
//    - Same mapping steps: convert pin to (TIMx, channel); channel unused
//      here.
    GPIO_TypeDef *port;
    unsigned int pin;
    ecPinmap(pinName, &port, &pin);
    TIM_TypeDef *TIMx;
    int chN;
    PWM_pinmap(pinName, &TIMx, &chN);

// 1. Set Counter Period in msec
//    - Calls base helper: sets PSC/ARR to achieve 'msec' update event period.
    TIM_period_ms(TIMx, msec);

}

// allowable range for msec: 1~2,000
void PWM_period(PinName_t pinName, uint32_t msec){
    PWM_period_ms(pinName, msec);           // Simple wrapper
for ms period API
}

```

```

// allowable range for usec: 1~1,000
void PWM_period_us(PinName_t pinName, uint32_t usec){

// 0. Match TIMx from Port and Pin
// - Resolve (TIMx, channel); channel unused for period-only operation.
GPIO_TypeDef *port;
unsigned int pin;
ecPinmap(pinName, &port, &pin);
TIM_TypeDef *TIMx;
int chN;
PWM_pinmap(pinName, &TIMx, &chN);

// 1. Set Counter Period in usec
// - Calls base helper: sets f_cnt to 1 MHz and ARR to 'usec'.
    TIM_period_us(TIMx, usec); //YOUR CODE GOES HERE // Already implemented
    by this call

}

/* DUTY RATIO SETUP */
// High Pulse width in msec
void PWM_pulsewidth(PinName_t pinName, double pulse_width_ms){
// 0. Match TIMx from Port and Pin
// - Resolve timer instance and channel to write the proper CCRx.
GPIO_TypeDef *port;
unsigned int pin;
ecPinmap(pinName, &port, &pin);
TIM_TypeDef *TIMx;
int chN;
PWM_pinmap(pinName, &TIMx, &chN);

// 1. Declaration System Frequency and Prescaler
// - fsys is scaled in kHz for ms-based math (84,000 for 84 MHz PLL; 16,000
for 16 MHz HSI).
// - psc is the current prescaler register value (actual divide is psc+1).
    uint32_t fsys = 0; // scaled system frequency in kHz (ms
path)
    uint32_t psc = TIMx->PSC; // TIM prescaler register

// 2. Check System CLK: PLL or HSI
// - Decide scaled fsys according to current system clock source.
    if((RCC->CFGR & RCC_CFGR_SW_PLL) == RCC_CFGR_SW_PLL) fsys = 84000;
// 84 MHz → 84,000 kHz
    else if((RCC->CFGR & RCC_CFGR_SW_HSI) == RCC_CFGR_SW_HSI) fsys = 16000;

```

```

// 16 MHz → 16,000 kHz

// 3. Configure prescaler PSC
// - fclk (kHz) = fsys(kHz)/(psc+1); CCR value (ticks) = pulse_width_ms *
fclk - 1.
// - Writing CCRx defines the high-time for PWM mode 1.
float fclk = fsys/(psc+1); // counter clock in kHz
uint32_t value = pulse_width_ms *fclk - 1; // CCR ticks for requested
ms

switch(chN){
    case 1: TIMx->CCR1 = value; break; // Write compare to channel
1
    case 2: TIMx->CCR2 = value; break; // Write compare to channel
2
    case 3: TIMx->CCR3 = value; break; // Write compare to channel
3
    case 4: TIMx->CCR4 = value; break; // Write compare to channel
4
    // REPEAT for CHn=2, 3, 4
    // REPEAT for CHn=2, 3, 4
    // REPEAT for CHn=2, 3, 4
    default: break; // Safety: unknown channel
}
}

// High Pulse width in msec
void PWM_pulsewidth_ms(PinName_t pinName, double pulse_width_ms){
    PWM_pulsewidth(pinName, pulse_width_ms); // Alias of PWM_pulsewidth
(ms units)
}

// High Pulse width in usec
void PWM_pulsewidth_us(PinName_t pinName, double pulse_width_us){
// 0. Match TIMx from Port and Pin
// - Resolve timer instance and corresponding channel to write CCRx.
GPIO_TypeDef *port;
unsigned int pin;
ecPinmap(pinName, &port, &pin);
TIM_TypeDef *TIMx;
int chN;
PWM_pinmap(pinName, &TIMx, &chN);

// 1. Declaration system frequency and prescaler
// - fsys is scaled in MHz for us-based math (84 or 16).

```

```

    uint32_t fsys = 0;                // scaled system frequency in MHz (us
path)
    uint32_t psc = TIMx->PSC;         // current prescaler value

// 2. Check System CLK: PLL or HSI
//    - Select 84 or 16 according to PLL/HSI, to keep math in MHz units.
    if((RCC->CFGR & RCC_CFGR_SW_PLL) == RCC_CFGR_SW_PLL)    fsys = 84;
// 84 MHz → 84 in MHz units
    else if((RCC->CFGR & RCC_CFGR_SW_HSI) == RCC_CFGR_SW_HSI) fsys = 16; //
16 MHz → 16 in MHz units

// 3. Configure prescaler PSC
//    - fclk (MHz) = fsys(MHz)/(psc+1); CCR ticks = pulse_width_us * fclk - 1.
//    - Write to the mapped CCRx register for the target channel.
    float fclk = fsys/(psc+1);        // counter clock in MHz
    uint32_t value = pulse_width_us * fclk - 1;    // CCR ticks for
requested us

    switch(chN){
        case 1: TIMx->CCR1 = value; break;    // Compare for CH1
        case 2: TIMx->CCR2 = value; break;    // Compare for CH2
        case 3: TIMx->CCR3 = value; break;    // Compare for CH3
        case 4: TIMx->CCR4 = value; break;    // Compare for CH4
        default: break;                      // Safety: unknown
channel
    }
}

// Duty Ratio from 0 to 1
void PWM_duty(PinName_t pinName, float duty){

// 0. Match TIMx from Port and Pin
//    - Resolve the timer and the target channel to write CCRx.
    GPIO_TypeDef *port;
    unsigned int pin;
    ecPinmap(pinName, &port, &pin);
    TIM_TypeDef *TIMx;
    int chN;
    PWM_pinmap(pinName, &TIMx, &chN);

// 1. Configure prescaler PSC
//    - Compute CCR target from duty: CCR = (ARR+1)*duty.
//    - Clamp to [0, ARR+1] to avoid underflow/overflow in edge cases.

```

```

    uint32_t value = ((TIMx->ARR)+1)*duty;    // duty in [0..1] scaled to
timer period

    if(duty <= 0.f)
        value = 0;                            // 0% duty ⇒ CCR = 0
    else if(duty >= 1.f)
        value = TIMx->ARR + 1;                // 100% duty ⇒ CCR = ARR+1
(effectively always high)

    // Write the computed compare value to the mapped channel
    if(chN == 1)    { TIMx->CCR1 = value; }
    else if(chN == 2) { TIMx->CCR2 = value; }
    else if(chN == 3) { TIMx->CCR3 = value; }
    else            { TIMx->CCR4 = value; } // CH4
    // REPEAT for CHn=2, 3, 4
    // REPEAT for CHn=2, 3, 4
    // REPEAT for CHn=2, 3, 4
}

// DO NOT MODIFY HERE
void PWM_pinmap(PinName_t pinName, TIM_TypeDef **TIMx, int *chN)
{
    GPIO_TypeDef *port;
    unsigned int pin;
    ecPinmap(pinName, &port, &pin);    // Resolve to (port, pin) first

    // Below: map concrete GPIO pins to a specific TIM instance and channel.
    // Each case sets *TIMx to the timer base address and *chN to the channel
    (1..4).

    if(port == GPIOA) {
        switch(pin){
            case 0 : *TIMx = TIM2; *chN = 1; break;    // PA0 → TIM2 CH1
            case 1 : *TIMx = TIM2; *chN = 2; break;    // PA1 → TIM2 CH2
            case 5 : *TIMx = TIM2; *chN = 1; break;    // PA5 → TIM2 CH1 (alt
function option on some variants)
            case 6 : *TIMx = TIM3; *chN = 1; break;    // PA6 → TIM3 CH1
            //case 7: TIMx = TIM1; *chN = 1N; break;    // PA7 → (commented example
for TIM1 CH1N, not used)
            case 8 : *TIMx = TIM1; *chN = 1; break;    // PA8 → TIM1 CH1
            case 9 : *TIMx = TIM1; *chN = 2; break;    // PA9 → TIM1 CH2
            case 10: *TIMx = TIM1; *chN = 3; break;    // PA10 → TIM1 CH3
            case 15: *TIMx = TIM2; *chN = 1; break;    // PA15 → TIM2 CH1 (remapped
function)
            default: break;                            // Other PA pins not mapped
for PWM here

```

```

    }
}
else if(port == GPIOB) {
    switch(pin){
        //case 0: TIMx = TIM1; *chN = 2N; break; // PB0 → (example for TIM1
CH2N), commented out
        //case 1: TIMx = TIM1; *chN = 3N; break; // PB1 → (example for TIM1
CH3N), commented out
        case 3 : *TIMx = TIM2; *chN = 2; break; // PB3 → TIM2 CH2
        case 4 : *TIMx = TIM3; *chN = 1; break; // PB4 → TIM3 CH1
        case 5 : *TIMx = TIM3; *chN = 2; break; // PB5 → TIM3 CH2
        case 6 : *TIMx = TIM4; *chN = 1; break; // PB6 → TIM4 CH1
        case 7 : *TIMx = TIM4; *chN = 2; break; // PB7 → TIM4 CH2
        case 8 : *TIMx = TIM4; *chN = 3; break; // PB8 → TIM4 CH3
        case 9 : *TIMx = TIM4; *chN = 4; break; // PB9 → TIM4 CH4
        case 10: *TIMx = TIM2; *chN = 3; break; // PB10 → TIM2 CH3
        default: break; // Other PB pins not mapped
here
    }
}
else if(port == GPIOC) {
    switch(pin){
        case 6 : *TIMx = TIM3; *chN = 1; break; // PC6 → TIM3 CH1
        case 7 : *TIMx = TIM3; *chN = 2; break; // PC7 → TIM3 CH2
        case 8 : *TIMx = TIM3; *chN = 3; break; // PC8 → TIM3 CH3
        case 9 : *TIMx = TIM3; *chN = 4; break; // PC9 → TIM3 CH4 (요청한
예: case 9 ... CH4, explained)
        default: break; // Other PC pins not mapped
here
    }
}
// TIM5 needs to be added, if used. // If your board routes PWM to TIM5
pins, extend cases above accordingly.
}

```

Procedure

Make a simple program that changes the angle of the RC servo motor that rotates back and forth from 0 deg to 180 degree within a given period of time.

Reset to '0' degree by pressing the push button (PC13).

- Button input has to be an External Interrupt

- Use Port A Pin 1 as PWM output pin for TIM2_CH2.
- Use Timer interrupt of period 500msec.
- Angle of RC servo motor should rotate from 0° to 180° and back 0° at a step of 10° at the rate of 500msec.

You need to observe how the PWM signal output is generated as the input button is pushed, using an oscilloscope. You need to capture the Oscilloscope output in the report.

1. Create a new project under the directory `\repos\EC\lab\LAB_PWM`

- The project name is “**LAB_PWM**”.
- Create a new source file named as “**LAB_PWM_RCmotor.c**”

You MUST write your name on the source file inside the comment section.

2. Include your updated library in `\repos\EC\include\` to your project.

- **ecPinNames.h ecPinNames.c**
- **ecGPIO.h, ecGPIO.c**
- **ecRCC.h, ecRCC.c**
- **ecEXTI.h, ecEXTI.c**
- **ecTIM.h, ecTIM.c**
- **ecPWM.h ecPWM.h**

3. Connect the RC servo motor to MCU pin (PA1) , VCC and GND

4. Increase the angle of RC servo motor from 0° to 180° with a step of 10° every 500msec. After reaching 180°, decrease the angle back to 0°. Use timer interrupt IRQ.

5. When the button is pressed, it should reset to the angle 0° and start over. Use EXT interrupt.

Configuration

Type	Port - Pin	Configuration
Button	Digital In (PC13)	Pull-Up
PWM Pin	AF (PA1)	Push-Pull, Pull-Up, Fast
PWM Timer	TIM2_CH2 (PA1)	TIM2 (PWM) period: 20msec, Duty ratio: 0.5~2.5msec
Timer Interrupt	TIM3	TIM3: Timer Interrupt of 500 msec

Algorithm

Overview

Mealy FSM Table — RC Servo Sweep Control (PA_1, TIM3 = 500 ms, EXTI on PC_13)

Current State	Input Condition	Next State	Outputs (PWM Duty / Angle)	Actions (Mealy)
S0: UP	tick_500ms & count < 180	S0	PWM = angle_to_ms(count)	count $\leftarrow$ count + 10
S0: UP	tick_500ms & count $\geq$ 180 (reaches upper limit)	S1	PWM = angle_to_ms(180)	count $\leftarrow$ 180; angle $\leftarrow$ -10
S0: UP	btn_fall (anytime)	S0	PWM = angle_to_ms(0)	count $\leftarrow$ 0; angle $\leftarrow$ +10 (executed inside EXTI ISR; clear pending)
S1: DOWN	tick_500ms & count > 0	S1	PWM = angle_to_ms(count)	count $\leftarrow$ count - 10
S1: DOWN	tick_500ms & count $\leq$ 0 (reaches lower limit)	S0	PWM = angle_to_ms(0)	count $\leftarrow$ 0; angle $\leftarrow$ +10
S1: DOWN	btn_fall (anytime)	S0	PWM = angle_to_ms(0)	count $\leftarrow$ 0; angle $\leftarrow$ +10 (executed inside EXTI ISR; clear pending)

Discussion

1. Derive a simple logic to calculate CRR and ARR values to generate x[Hz] and y[%] duty ratio of PWM. How can you read the values of input clock frequency and PSC?

A. Core formulas (edge-aligned, up-counting)

Let

- f_{timclk} = timer input clock to TIMx (after APB & PLL, including the “ $\times 2$ ” rule),
- PSC = prescaler register value (hardware divides by PSC+1),
- ARR = auto-reload register (period counts - 1),
- CCR = capture/compare register (on the channel you use),
- x = desired PWM frequency [Hz],

- y = desired duty ratio [%] (0–100).

Then:

```
f_cnt = f_timclk / (PSC + 1) // timer counter clock (tick rate)
x = f_cnt / (ARR + 1) // PWM frequency  ARR = round( f_cnt / x ) - 1
CCR = round( (ARR + 1) * (y / 100) ) // 0 ≤ CCR ≤ ARR
```

Notes

- Some libraries treat `ARR` as “period–1” and `CCR` as the 0...ARR compare value. The formulas above match that convention.
- For center-aligned or different PWM modes, the period relationship changes; the logic above is for **edge-aligned, up-counting** (most common in labs).

B. Practical “choose PSC first, then ARR”

1. Pick a convenient `PSC` to keep `ARR` in a good numeric range (e.g., 200–100 000) for resolution.
2. Compute:

```
f_cnt = f_timclk / (PSC + 1)  ARR = round( f_cnt / x ) - 1  CCR = round( (ARR + 1) *
y / 100 )
```

3. Program `TIMx->PSC = PSC`, `TIMx->ARR = ARR`, `TIMx->CCRx = CCR`.

C. How to read timer input clock `f_timclk`

- Get the APB bus clocks and apply the STM32 “×2 when APB prescaler > 1” rule:
 - For **APB1 timers** (TIM2,3,4,5...): `PCLK1 = HAL_RCC_GetPCLK1Freq()` // or compute from `RCC->CFGR` if `(APB1 prescaler == 1)` `f_timclk = PCLK1`; else `f_timclk = 2 * PCLK1`;
 - For **APB2 timers** (TIM1,8,9,10,11): `PCLK2 = HAL_RCC_GetPCLK2Freq()` if `(APB2 prescaler == 1)` `f_timclk = PCLK2`; else `f_timclk = 2 * PCLK2`;

In many lab setups on F411RE: APB1 = 42 MHz, APB1 prescaler > 1 → **timer clock on APB1 timers is 84 MHz**. APB2 often 84 MHz; if prescaler > 1, timer clock becomes 168 MHz. Always confirm your clock tree.

D. How to read **PSC** at runtime

- `uint32_t psc_val = TIMx->PSC`;
- The effective division is `PSC+1`, so the actual prescaler factor is `psc_val + 1`.

E. Quick numeric example

Suppose TIM3 on APB1 with $f_{timclk} = 84 \text{ MHz}$, want $x = 10 \text{ kHz}$, $y = 25\%$:

Choose $PSC = 0 \rightarrow f_{cnt} = 84,000,000 / (0+1) = 84,000,000 \text{ Hz}$ $ARR = \text{round}(84,000,000 / 10,000) - 1 = 8400 - 1 = 8399$ $CCR = \text{round}((8399+1) * 0.25) = \text{round}(8400 * 0.25) = 2100$

Program:

```
TIM3->PSC = 0; TIM3->ARR = 8399; TIM3->CCR1 = 2100; // if using CH1
```

2. What is the smallest and highest PWM frequency that can be generated for Q1?

Let timer resolution be N-bit (most STM32 general timers are **16-bit**: PSC and ARR are 0... 65,535; TIM2/TIM5 are 32-bit).

With edge-aligned, up-counting:

- **Highest PWM frequency**

Achieved when the period is minimal. Practically, use $PSC = 0$, $ARR = 1$ (since $ARR = 0$ leaves no useful duty resolution). $f_{pwm,max} \approx f_{timclk} / (ARR+1) = f_{timclk} / 2$ If you absolutely allow $ARR = 0$, then $f_{pwm} = f_{timclk}$, but CCR can only be 0 (or 1 depending on mode), i.e., no meaningful duty control. So use **$ARR \geq 1$** for real PWM.

- **Smallest PWM frequency**

Achieved with the largest total division. For **16-bit PSC and 16-bit ARR**: $PSC = 65535$, $ARR = 65535$ $f_{pwm,min} = f_{timclk} / [(PSC+1) * (ARR+1)] = f_{timclk} / (65,536 * 65,536)$ Example with $f_{timclk} = 84 \text{ MHz}$: $f_{pwm,min} \approx 84e6 / 4.294967296e9 \approx 0.0196 \text{ Hz}$ (≈ 51 seconds per period) If you use **TIM2/TIM5** (32-bit ARR), the minimum frequency can be orders of magnitude lower.

Bottom line (16-bit timers):

- Practical **max** $\approx f_{timclk} / 2$ (with $ARR = 1$, $PSC = 0$).
- Practical **min** $\approx f_{timclk} / (2^{16} * 2^{16})$ when both PSC and ARR are maxed out.
- Real designs pick PSC to put ARR in a range that gives good duty resolution and an exact or near-exact x .

Code

Your code goes here: https://github.com/henny041520-commits/EC-DJLee-042/blob/main/lab/LAB_PWM/LAB_PWM_RCmotor.c

Explain your source code with necessary comments.

```
/**
```

```
*****
```

```

* Function: Sweep an RC servo on PA_1 between 0°↔180°
*           - TIM3 update interrupt (every 500 ms) adjusts the target angle
*           - EXTI on PC_13 (falling edge) resets servo immediately to 0°
*****
*/
#include "ecSTM32F4v2.h" // EC course HAL: RCC, GPIO, EXTI, SysTick, PWM,
TIM, pin map

/* ----- Pin/servo constants ----- */
#define RC_PWM    PA_1 // Servo signal pin (mapped internally to a TIMx
channel)
#define Button    PC_13 // User button (EXTI source, falling edge)
#define mindeg    0.5 // PWM high-time [ms] at 0° (typical servo ~0.5
ms)
#define maxdeg    2.5 // PWM high-time [ms] at 180° (typical servo ~2.5
ms)
#define stepduty  2.0 // (unused) example constant if stepping duty
directly

/* ----- Motion control variables -----
* 'count' holds the current target angle [deg] that we convert to pulse
width.
* 'angle' is the signed step (±10 deg). It flips sign at 0°/180° to sweep
back.
*/
int angle = 10; // +10° when going up, -10° when going down
int count = 0; // current target angle in degrees (0..180)

/* Forward declarations */
void setup(void);

/* Convert angle(deg) → high pulse width in milliseconds for standard servo.
* Clamps input to [0,180], then linearly maps to [mindeg, maxdeg].
* Used from both ISR contexts (TIM3 and EXTI).
*/
static inline float angle_to_ms(int deg){
    if(deg < 0)    deg = 0;
    if(deg > 180) deg = 180;
    return mindeg + ((maxdeg - mindeg) * ((float)deg / 180.0f));
}

int main(void) {
    /* ----- System / peripheral init -----
    * - System clock: 84 MHz via PLL (required by EC HAL timing presets)
    * - SysTick: 1 ms tick (HAL delays, not used directly here)
    * - GPIO/PWM: PA_1 as TIMx PWM output, 20 ms period (50 Hz servo)

```

```

    * - TIM3_UI: update event every 500 ms (sweep step timing)
    * - EXTI: PC_13 falling edge → immediate reset to 0°
    */
    setup();

    /* ----- Foreground loop -----
    * Nothing to do in main; control is interrupt-driven:
    * - TIM3_IRQHandler(): periodic angle update + PWM refresh
    * - EXTI15_10_IRQHandler(): button reset + PWM refresh
    */
    while(1){
        /* idle / WFI-friendly loop; all work done in ISRs */
    }
}

/* ----- Board & peripheral configuration ----- */
void setup(void)
{
    RCC_PLL_init();           // System clock = 84 MHz (APB clocks set in RCC
HAL)
    SysTick_init();           // 1 ms system tick (for delay_ms() etc.)

    /* ---- GPIO & PWM output on PA_1 ---- */
    GPIO_init(RC_PWM, AF);    // PA_1 to Alternate Function (timer
output)
    GPIO_pupd(RC_PWM, pullup); // Pull-up (keeps line defined if timer
off)
    GPIO_ospeed(RC_PWM, fastspeed); // Fast slew for clean edges
    GPIO_otype(RC_PWM, pushpull); // Push-pull output

    PWM_init(RC_PWM);         // Route PA_1 to its TIM channel, set PWM
mode
    PWM_period(RC_PWM, 20);    // Servo period = 20 ms (≈50 Hz)

    /* ---- Push-button on PC_13 with EXTI ---- */
    GPIO_init(Button, INPUT); // PC_13 as input
    GPIO_pupd(Button, pullup); // Pull-up; button press → falling edge
    EXTI_init(Button, FALL, 2); // EXTI on falling edge, NVIC prio=2

    /* ---- TIM3 periodic update (500 ms) ----
    * TIM_UI_init configures timer, enables update interrupt & NVIC.
    * Then we set its period explicitly to 500 ms and enable UI.
    */
    TIM_UI_init(TIM3, 500);    // Base init + NVIC enable
    TIM_period_ms(TIM3, 500);  // Update event every 500 ms
    TIM_UI_enable(TIM3);       // Enable update interrupt generation

```

```

}

/* ----- TIM3 Update ISR -----
 * Called every 500 ms (when UIF is set).
 * 1) Write current target to PWM (Mealy-style immediate output)
 * 2) Advance target by 'angle' ( $\pm 10^\circ$ )
 * 3) Clamp & flip direction at boundaries ( $0^\circ$  and  $180^\circ$ )
 * 4) Clear UIF
 */
void TIM3_IRQHandler(void){
    if(is_UIF(TIM3)){ // Update event pending?
        PWM_pulsewidth_ms(RC_PWM, angle_to_ms(count)); // Apply current target
        count += angle; // Move target by step

        if (count >= 180) { // Upper limit reached
            count = 180; // Clamp
            angle = -10; // Reverse direction
        }
        if (count <= 0) { // Lower limit reached
            count = 0; // Clamp
            angle = 10; // Reverse direction
        }

        clear_UIF(TIM3); // Acknowledge interrupt
    }
}

/* ----- EXTI ISR: PC_13 (lines 15..10) -----
 * Falling-edge button press → immediate reset:
 * 1) count=0, angle=+10 (prepare to sweep upward)
 * 2) Update PWM to  $0^\circ$  right away (Mealy output)
 * 3) Clear EXTI pending
 */
void EXTI15_10_IRQHandler(void){
    if(is_pending_EXTI(Button)){ // Is EXTI from PC_13?
        count = 0; // Reset target angle
        angle = 10; // Next motion: upward
        PWM_pulsewidth_ms(RC_PWM, angle_to_ms(0)); // Drive servo to  $0^\circ$ 
        clear_pending_EXTI(Button); // Clear EXTI pending
    }
}

```

Example Code

Sample Code : Timer Interrupt IRQ

```

#include "stm32f411xe.h"
#include "ecGPIO.h"
#include "ecRCC.h"
#include "ecTIM.h"

#define LED_PIN 5
uint32_t _count = 0;
void setup(void);

int main(void) {
    // Initialization -----
    setup();

    // Infinite Loop -----
    while(1){}
}

// Initialization
void setup(void){
    RCC_PLL_init();           // System Clock = 84MHz
    GPIO_init(GPIOA, LED_PIN, OUTPUT); // calls RCC_GPIOA_enable()
    TIM_UI_init(TIM2, 1);     // TIM2 Update-Event Interrupt every 1
msec
    TIM_UI_enable(TIM2);
}

void TIM2_IRQHandler(void){
    if(is_UIF(TIM2)){         // Check UIF(update interrupt flag)
        _count++;
        if (_count > 1000) {
            LED_toggle();     // LED toggle every 1 sec
            _count = 0;
        }
        clear_UIF(TIM2);     // Clear UI flag by writing 0
    }
}

```

Sample Code : PWM output

```

#include "stm32f411xe.h"
#include "math.h"

```

```

// #include "ecSTM32F411.h"
#include "ecPinNames.h"
#include "ecGPIO.h"
#include "ecSysTick.h"
#include "ecRCC.h"
#include "ecTIM.h"
#include "ecPWM.h"    // ecPWM2.h

// Definition Button Pin & PWM Port, Pin
#define BUTTON_PIN 13
#define PWM_PIN PA_5
void setup(void);

int main(void) {
    // Initialization -----
    setup();

    // Infinite Loop -----
    while(1){
        LED_toggle();
        for (int i=0; i<5; i++) {
            PWM_duty(PWM_PIN, (float)0.2*i);
            delay_ms(1000);
        }
    }
}

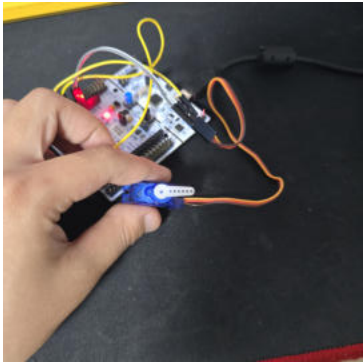
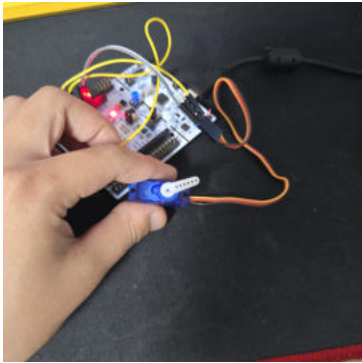
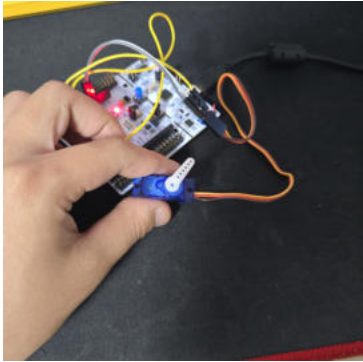
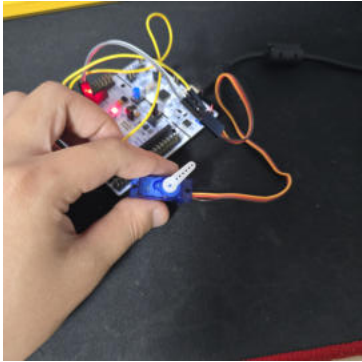
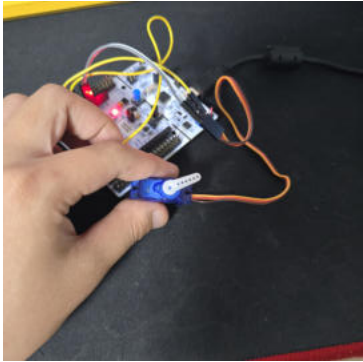
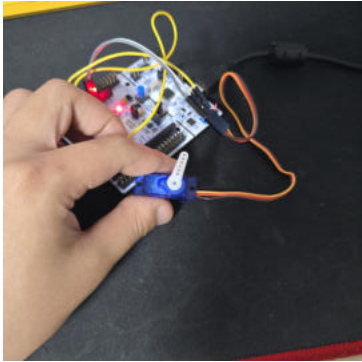
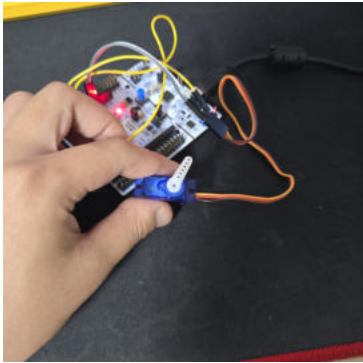
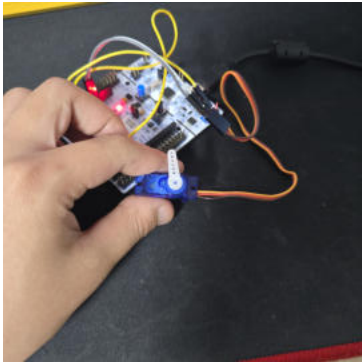
// Initialiization
void setup(void) {
    RCC_PLL_init();
    SysTick_init();

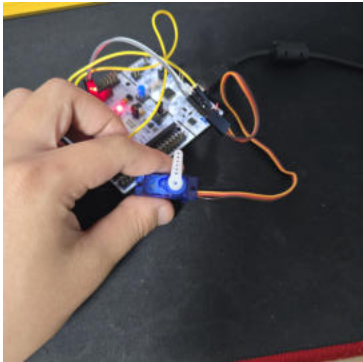
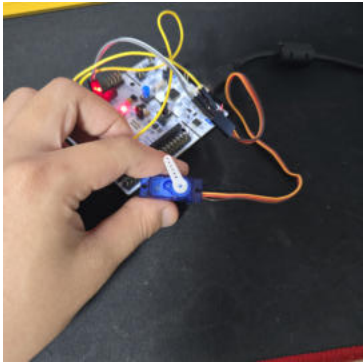
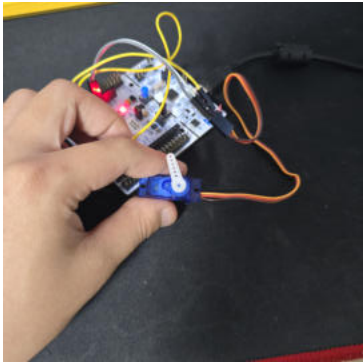
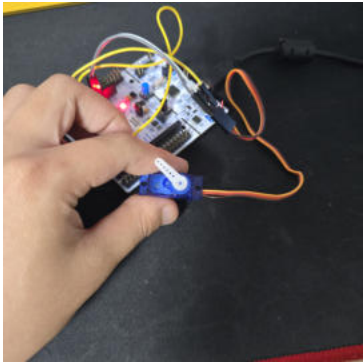
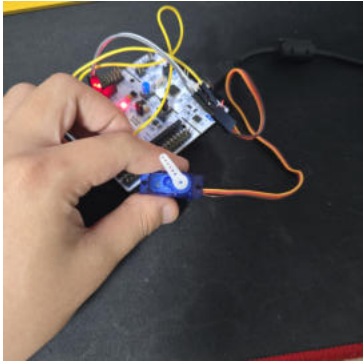
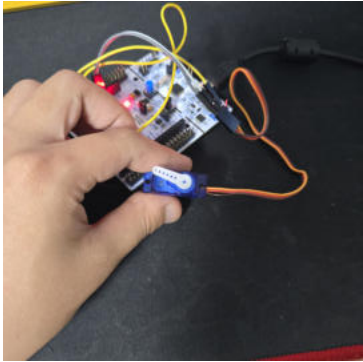
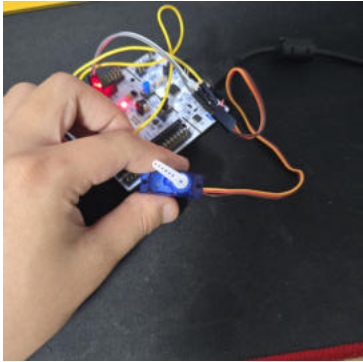
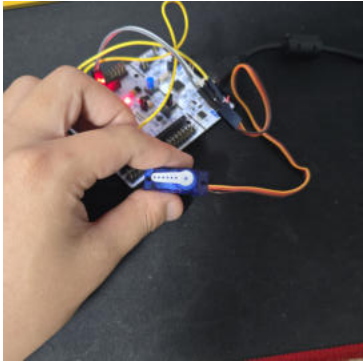
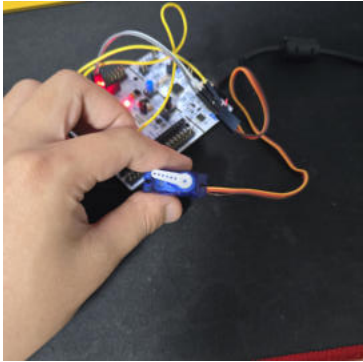
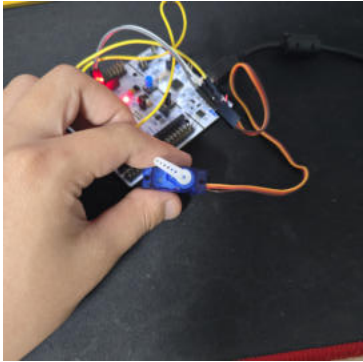
    // PWM of 20 msec:  TIM2_CH1 (PA_5 AFmode)
    GPIO_init(GPIOA, 5, EC_AF);
    PWM_init(PWM_PIN);
    PWM_period(PWM_PIN, 20);    // 20 msec PWM period
}

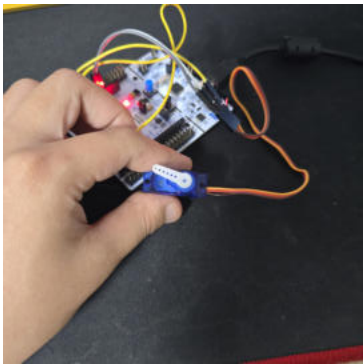
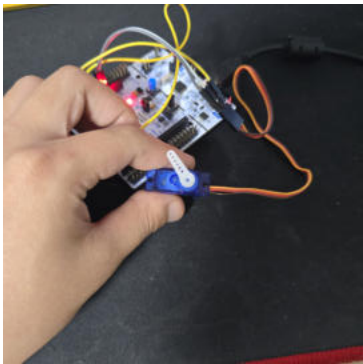
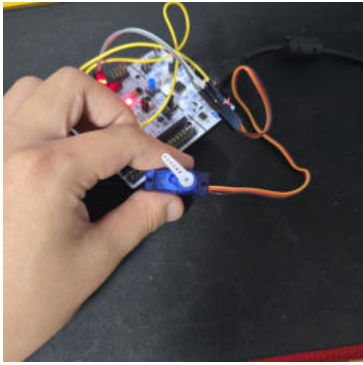
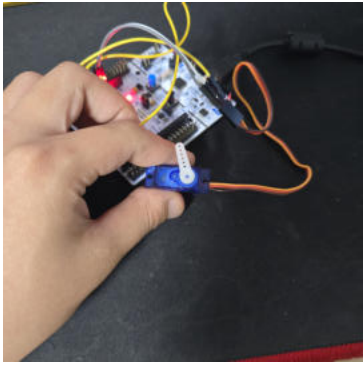
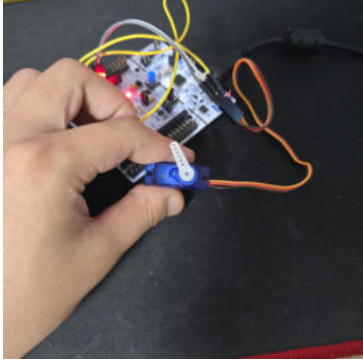
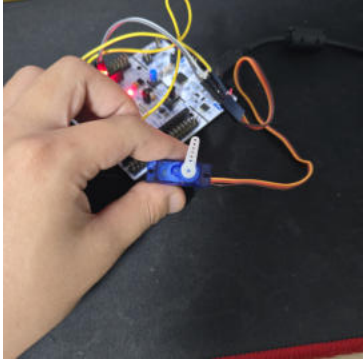
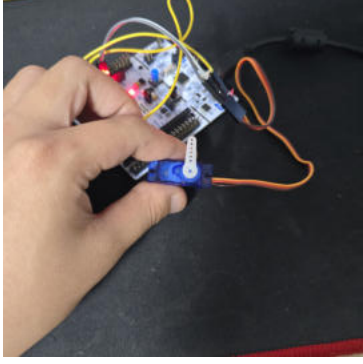
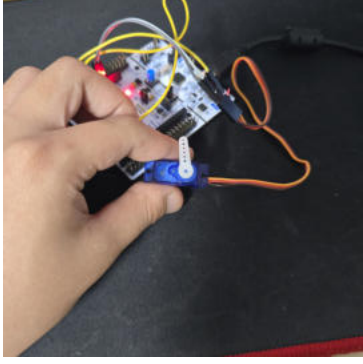
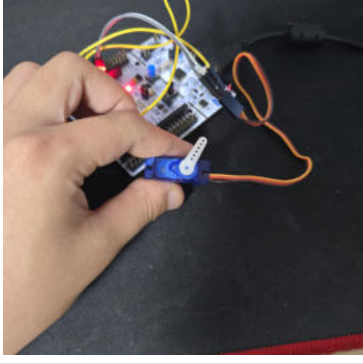
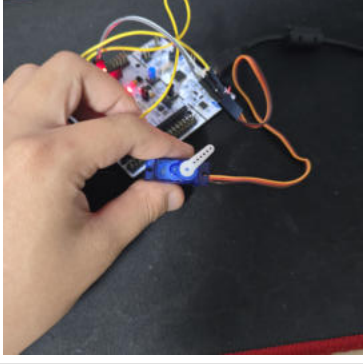
```

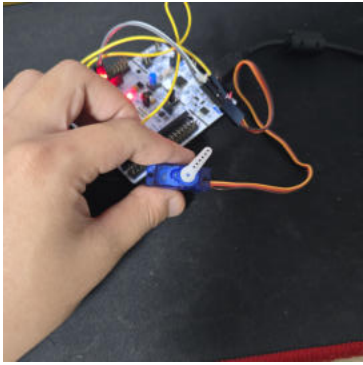
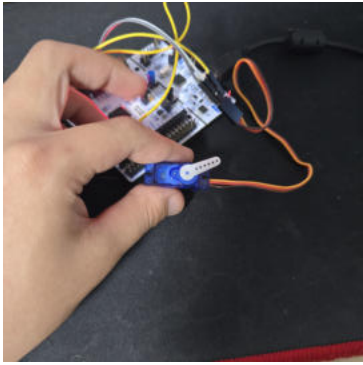
Results

Experiment images and results

No	Image	No	Image
0		1	
2		3	
4		5	
6		7	

No	Image	No	Image
8		9	
10		11	
12		13	
14		15	
16		17	

No	Image	No	Image
18		19	
20		21	
22		23	
24		25	
26		27	

No	Image	No	Image
28		29	

<https://youtube.com/shorts/VR7RJFKOLqk?feature=share>

Problem 2: DC motor

Procedure

Make a simple program that rotates a DC motor that changes the duty ratio from 25% -->75%--> 25% --> and so on.

The rotating speed level changes every 2 seconds.

By pressing the push button (PC13), toggle from Running and stopping the DC motor

First, you MUST read [Tutorial: DC motor driver connection](#)

1. Use the same project.
 - Create a new source file named “**LAB_PWM_DCmotor.c**”
 - You need to eliminate the other source file that contains `main()` from the project
 - e.g. Eliminate “**LAB_PWM_RCmotor.c**” from the project

You MUST write your name on the source file inside the comment section.

2. Connect DC motor and DC motor driver.
 - PA_0 for the DC motor PWM
 - PC_2 for Direction Pin

3. Change DC motor from LOW Speed to HIGH Speed for every 2 seconds
 - e.g. 25% -->75%--> 25% --> and so on.

4. When Button is pressed, it should PAUSE or CONTINUE motor run

Configuration

Function	Port - Pin	Configuration
Button	Digital In (PC13)	Pull-Up
Direction Pin	Digital Out (PC2)	Push-Pull
PWM Pin	AF (PA0)	Push-Pull, Pull-Up, Fast
PWM Timer	TIM2_CH1 (PA0)	TIM2 (PWM) period: 1msec (1kHz)
Timer Interrupt	TIM3	TIM3: Timer Interrupt of 500 msec

Algorithm

Overview

Mealy FSM Table (edge-triggered by TIM3_IRQHandler tick = 1 ms, and EXTI button fall)

Current State	Input	Condition	Next State	Outputs	Actions (Mealy)
S0: RUN_75	tick_1ms	count+1 < 4	S0	PWM_duty(DC_PWM, 0.75)	count ← count+1
S0: RUN_75	tick_1ms	count+1 ≥ 4	S1	PWM_duty(DC_PWM, 0.25)	duty_state ← duty_state ⊕ 1 (0→1); count ← 0
S0: RUN_75	btn_fall	—	S2	PWM_duty(DC_PWM, 1.0)	run_toggle ← 0 (via run_toggle ⊕=1); count unchanged
S1: RUN_25	tick_1ms	count+1 < 4	S1	PWM_duty(DC_PWM, 0.25)	count ← count+1
S1: RUN_25	tick_1ms	count+1 ≥ 4	S0	PWM_duty(DC_PWM, 0.75)	duty_state ← duty_state ⊕ 1 (1→0); count ← 0

Current State	Input	Condition	Next State	Outputs	Actions (Mealy)
S1: RUN_25	btn_fall	—	S2	PWM_duty(DC_PWM, 1.0)	run_toggle ← 0; count unchanged
S2: PAUSE_100	tick_1ms	—	S2	PWM_duty(DC_PWM, 1.0)	— (no count++)
S2: PAUSE_100	btn_fall	duty_state = 0	S0	PWM_duty(DC_PWM, 0.75)	run_toggle ← 1; count unchanged
S2: PAUSE_100	btn_fall	duty_state = 1	S1	PWM_duty(DC_PWM, 0.25)	run_toggle ← 1; count unchanged

Your code goes here: https://github.com/henny041520-commits/EC-DJLee-042/blob/main/lab/LAB_PWM/LAB_PWM_DCmotor.c

Explain your source code with necessary comments.

```
/**
*****
* Behavior summary:
* - PWM on PA_0 drives a DC motor/actuator.
* - While "running", duty alternates between 75% and 25% after a fixed number
  of timer ticks.
* - Button (PC_13, falling edge) toggles between RUN and PAUSE.
*   - PAUSE forces duty to 100% (full on).
*****
*/
#include "ecSTM32F4v2.h"
#include "ecTIM2.h"
#include "ecPWM2.h"

#define DC_PWM    PA_0    // PWM output pin (mapped to a TIMx channel via
                          // ecHAL pinmap)
#define Button    PC_13   // User button input (EXTI source)
#define Direc     PC_2    // Direction GPIO (set once to HIGH here)

/* --- State variables --- */
static int run_toggle = 1; // 1: RUN mode (duty toggles 75%/25%), 0: PAUSE
                          // mode (force 100%)
static int duty_state = 0; // 0: 75% duty, 1: 25% duty
static int count = 0;      // tick counter used to time the duty toggle
                          // window
```

```

/* Map duty_state -> actual PWM duty write */
void apply_duty_from_state(void){
    if(duty_state == 0) PWM_duty(DC_PWM, 0.75f); // 75% duty
    else                 PWM_duty(DC_PWM, 0.25f); // 25% duty
}

/* One-time board/peripheral setup */
void setup(void){
    RCC_PLL_init(); // System clock configuration (e.g.,
84 MHz)
    SysTick_init(); // 1 ms systick (delays, timebase)

    /* Button: PC_13 input with pull-up, EXTI on falling edge */
    GPIO_init(Button, INPUT);
    GPIO_pupd(Button, pullup);
    EXTI_init(Button, FALL, 1); // EXTI priority = 1

    /* Direction pin: PC_2 push-pull output, set HIGH once */
    GPIO_init(Direc, OUTPUT);
    GPIO_otype(Direc, pushpull);
    GPIO_write(Direc, 1);

    /* PWM output: PA_0 set to AF, configure electrical traits */
    GPIO_init(DC_PWM, AF);
    GPIO_ospeed(DC_PWM, fastspeed);
    GPIO_otype(DC_PWM, pushpull);
    GPIO_pupd(DC_PWM, pullup);

    /* PWM: initialize and set base period */
    PWM_init(DC_PWM); // configures TIMx channel in PWM
mode
    PWM_period(DC_PWM, 1); // period = 1 ms (edge-aligned)

    /* Initialize run/duty state and apply initial duty (75%) */
    duty_state = 0;
    run_toggle = 1;
    apply_duty_from_state();

    /* Timer update interrupt (TIM3) for periodic duty timing */
    TIM_UI_init(TIM3, 500); // sets up TIM3 UI IRQ (period setup
depends on HAL)
    TIM_UI_enable(TIM3); // enable update interrupt
generation
}

```

```

int main(void){
    setup();
    while(1){
        __WFI();
EXTI)
    }
}

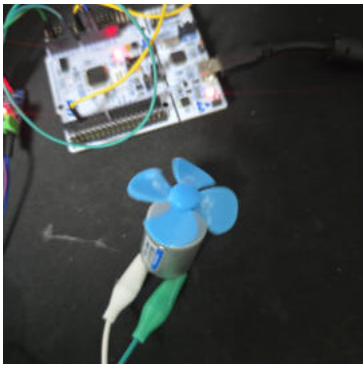
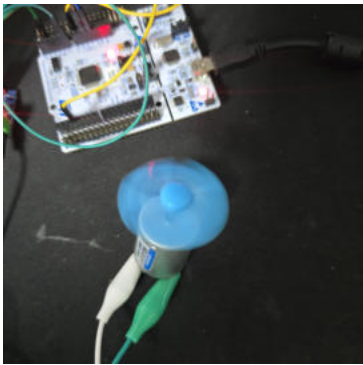
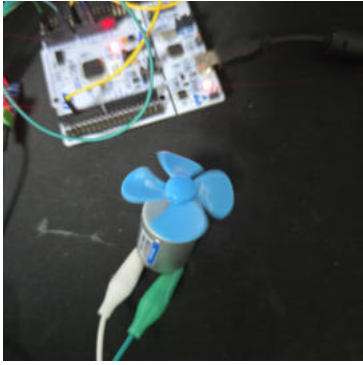
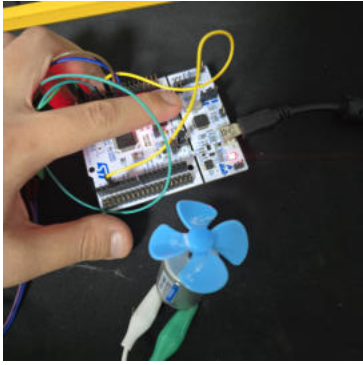
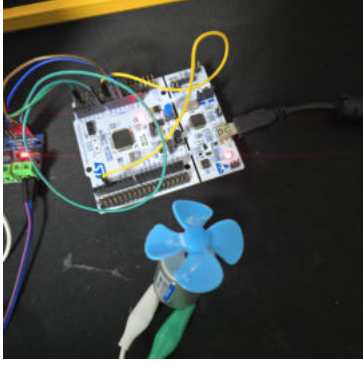
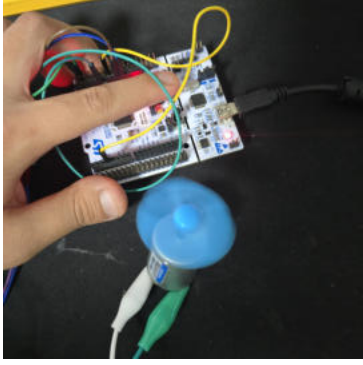
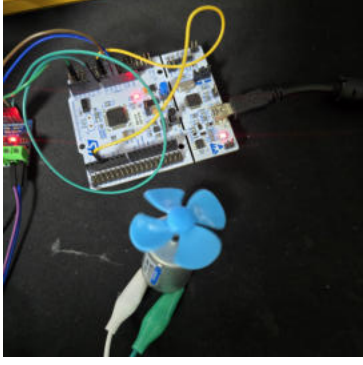
/* TIM3 update interrupt handler
 * - RUN mode: increment count, every 4 ticks toggle duty 75% <-> 25%, then
write PWM
 * - PAUSE mode: force duty to 100%
 */
void TIM3_IRQHandler(void){
    if(is_UIF(TIM3)){
        if(run_toggle){
            count++;
            if(count >= 4){
                duty_state ^= 1;
                count = 0;
            }
            apply_duty_from_state();
        } else {
            PWM_duty(DC_PWM, 1);
        }
        clear_UIF(TIM3);
    }
}

/* EXTI handler for PC_13 (lines 15..10)
 * - Toggle run/paused.
 * - On pause: force 100% duty; on resume: apply current duty_state (75% or
25%).
 */
void EXTI15_10_IRQHandler(void){
    if(is_pending_EXTI(Button)){
        run_toggle ^= 1;
        if(run_toggle == 0){
            PWM_duty(DC_PWM, 1);
        } else {
            apply_duty_from_state();
        }
        clear_pending_EXTI(Button);
    }
}

```

Results

Experiment images and results

No	Image	No	Image
0		1	
2		3	
4		5	
6		7	

Reference

Complete list of all references used (github, blog, paper, etc)

Troubleshooting

1. motor PWM duty ratio for different DIR

When, DIR=0 duty=0.8--> PWM 0.8 // 실제 모터에 전달되는 pwm

When, DIR=1 duty=0.8--> PWM 0.2 // 실제 모터에 전달되는 PWM

*** a solution ***

```
float targetPWM; // pwm for motor input
float duty=abs(DIR-targetPWM); // duty with consideration of DIR=1 or 0

PWM_duty(PWM_PIN, duty);
```

2. Motor does not run under duty 0.5

SOL) Configure motor PWM period as 1kHz

3. Check and give different Interrupt Priority

Check if you have different NVIC priority number for each IRQs

(Option) You can write Troubleshooting section

```
### 4. Print a string for BT (USART1)
Use `sprintf()`

```c++
#define _CRT_SECURE_NO_WARNINGS // sprintf 보안 경고로 인한 컴파일 에러 방지
#include <stdio.h> // sprintf 함수가 선언된 헤더 파일

char BT_string[20]=0;

int main()
{
```

```
 sprintf(BT_string, "DIR:%d PWM: %0.2f\n", dir, duty); // 문자, 정수, 실수
를 문자열로 만들
 USART1_write(BT_string, 20);
 // ...
}
```

<https://dojang.io/mod/page/view.php?id=352> \*\*

---